EXECUTIVE SUMMARY 3
1. PROJECT BACKGROUND & PROBLEM STATEMENT 4
1.1 Context 4
1.2 Problem Statement 4
1.3 Project Objectives 5
2. SYSTEM ARCHITECTURE & TECH STACK 7
2.1 Technology Stack & Architectural Justification 7
2.2 Backend Application Layer (Python / Flask) 7
2.3 Data Persistence Layer (MongoDB / PyMongo) 7
2.4 Presentation & Frontend Layer (HTML5, CSS3, Bootstrap 5) 7
2.4 Authentication & Security Layer (Werkzeug & Custom RBAC) 8
2.5 Reporting & In-Memory Data Streaming (csv & io.StringIO) 8
2.6 Tech Stack Summary 8
3. CORE MODULES & IMPLEMENTATION DETAILS 10
3.1 Role-Based Access Control (RBAC) & Security Architecture 10
3.2 Administrative Triage & Analytics Command Center 11
3.3 Activity Log & Milestone Timeline Architecture 12
3.4 Citizen Satisfaction Feedback Loop Architecture 13
4. DATABASE DESIGN & DATA MODEL 14
4.1 Collection: users 14
Document Schema (users) 14
Schema Field Definitions 14
4.2 Collection: requests 15
Document Schema (requests) 15
Schema Field Definitions 16
5. USER INTERFACE & DESIGN SYSTEM 18
5.1 Palette & Global Styling 18
5.2 Role-Specific Visual Branding 18
5.3 Overlay Modal Architecture 19

6. VERIFICATION & TESTING SUMMARY 20
7. FUTURE WORK & SCALABILITY ROADMAP 22
8. CONCLUSION 23

EXECUTIVE SUMMARY

The Public Service Request System is a modern, lightweight, and responsive web-based digital governance platform that I designed, architected, and developed to bridge the communication gap between municipal service providers and local citizens. The primary objective driving the development of this application was to replace slow, opaque, and traditional civic complaint channels with an accessible digital portal capable of facilitating transparent incident reporting, real-time status tracking, and streamlined administrative triage for municipal infrastructure issues—such as road defects, water distribution disruptions, solid waste accumulation, and public lighting failures.

During my initial domain analysis, I identified a pervasive structural problem in public service management where citizens often lodge complaint reports into an operational black box, receiving minimal to no visibility regarding whether their issue has been triaged, dispatched to repair crews, or resolved. To solve this problem of limited operational transparency, I engineered three core architectural components within the application. First, I implemented a request activity timeline and status history log, which acts as an immutable, event-driven audit stream embedded directly within each service ticket to record every state transition from initial pending triage to in-progress dispatch and final resolved closure, complete with administrative notes and precise timestamps. Second, I designed an administrative command center featuring a centralized triage dashboard equipped with live quantitative summary analytics, multi-parameter regex search filtering, and on-demand CSV data streaming capabilities to give municipal teams clear oversight of active infrastructure workloads. Third, I incorporated a citizen satisfaction feedback loop, serving as a post-resolution quality assurance mechanism that empowers citizens to evaluate completed municipal repairs using a one-to-five-star rating system accompanied by qualitative feedback comments.

To achieve high technical reliability, maintainability, and rapid development iteration, I built the application's core backend using Python and the Flask micro-framework, organized cleanly through a Blueprint-based architecture separating authentication, citizen workflows, and administrative operations. For data persistence, I integrated MongoDB using PyMongo, leveraging its flexible document model to handle schema evolution—specifically nested array updates for activity trails and feedback entries—without database migration overhead. Security and privacy were paramount throughout the implementation, leading me to establish strict, session-based Role-Based Access Control enforced through custom Python decorator functions that isolate administrative actions, prevent unauthorized privilege escalation across endpoints, and ensure smooth user navigation through intelligent, context-aware redirects. Overall, the system demonstrates how modern full-stack web engineering principles can be applied to build scalable, accountable, and citizen-centric municipal software solutions.

1. PROJECT BACKGROUND & PROBLEM STATEMENT

1.1 Context

In rapidly growing urban centers, effective municipal governance and public utility management depend heavily on direct, reliable, and transparent communication channels between residents and local government authorities. Municipalities are tasked with managing vast physical infrastructure networks, ranging from road maintenance and street lighting to water supply systems and solid waste collection. However, many municipal administrations continue to rely on traditional, manual complaint-handling mechanisms, such as walk-in service desks, paper-based incident forms, and central phone hotlines. In my review of typical municipal service workflows, I observed that these legacy channels create severe operational bottlenecks for both local government staff and the communities they serve. When citizens submit service requests through fragmented or paper-reliant systems, reports are easily misplaced, delayed in routing, or buried in unorganized backlogs, leading to low administrative accountability and poor resource allocation.

From the citizen's perspective, the lack of digital tracking transforms public service delivery into an opaque experience. Residents who report critical infrastructure issues—such as active water pipe bursts, hazardous road potholes, or prolonged power outages—are rarely provided with reference tracking numbers or progress updates. This absence of real-time communication forces citizens to repeatedly make follow-up phone calls or physical visits to municipal offices simply to check the status of a single request. This dynamic not only increases public frustration and erodes trust in local governance, but it also imposes an unnecessary administrative burden on municipal staff who must manually search through records to answer routine status inquiries. As urban populations expand, the volume of service requests naturally increases, making it imperative for local authorities to adopt centralized, web-based reporting systems that streamline administrative dispatch workflows while keeping citizens informed at every step of the resolution lifecycle.

1.2 Problem Statement

During my initial analysis of existing public service request workflows and municipal intake channels, I identified a series of structural inefficiencies that severely hinder effective service delivery. The primary issue stems from the disconnect between public expectations for prompt infrastructure repairs and the operational realities faced by municipal administrative teams. Through my evaluation of standard municipal complaint handling, I pinpointed three critical operational bottlenecks that negatively impact both citizens and municipal operations:

First, there is a pervasive lack of transparency in request tracking. When citizens submit a service report—whether regarding a damaged roadway, broken streetlamp, or blocked drainage system—the traditional workflow provides no automated or real-time progress updates. Once a

report is submitted, it enters an operational black box where citizens have no visibility into whether their issue has been logged, assigned to a maintenance crew, or scheduled for repair. This lack of feedback leaves citizens uncertain about the status of public infrastructure in their neighborhood, leading to duplicated reports for the same issue and increasing citizen frustration with local governance.

Second, administrative teams face severe operational triage bottlenecks. Without a centralized digital command dashboard, municipal staff must manually sort through incoming complaints arriving from various disjointed sources. Administrative personnel lack real-time data analytics, dynamic search utilities, and status-based filtering tools necessary to categorize incoming issues by severity or geographic location. As a result, prioritizing urgent repairs becomes inefficient, daily work queues become cluttered, and decision-makers are unable to quickly generate summary reports or export actionable data for field dispatch teams and departmental audits.

Third, there is a complete absence of accountability and feedback loops. In conventional systems, once an administrative worker or field technician marks a service ticket as completed or closed in an internal ledger, the workflow terminates immediately. There is no structured mechanism in place to notify the affected citizen or capture public feedback regarding the actual quality, completeness, or timeliness of the work performed. Without a post-resolution verification process, municipal supervisors cannot accurately evaluate field crew performance, identify recurring service quality issues, or ensure that reported problems have truly been resolved to the community's satisfaction.

1.3 Project Objectives

To address the operational bottlenecks identified in traditional public service workflows, I established a clear set of engineering objectives aimed at designing, developing, and deploying a robust digital solution. The overarching goal of the Public Service Request System is to create an accountable, transparent, and efficient bridge between community members and municipal administrators. To achieve this, I focused on four primary implementation objectives:

First, I aimed to develop a unified, role-segregated portal built on a responsive web architecture. My objective was to construct distinct user interfaces tailored specifically to the operational needs of citizens and administrators. For citizens, I designed an accessible dashboard focused on intuitive request submission, media upload capabilities, and status monitoring. For municipal staff, I developed an executive command center tailored for high-volume triage, queue management, and request resolution. To safeguard these portals, I implemented strict session-based Role-Based Access Control (RBAC) using custom Python decorators to ensure complete security isolation, prevent unauthorized cross-portal navigation, and maintain strict data privacy.

Second, I set out to implement a real-time, event-driven tracking system to eliminate the opacity of conventional complaint handling. I designed an embedded audit trail (activity_log) within the database structure to automatically capture every status change throughout a request's lifecycle—

transitioning from initial *Pending* submission to *In Progress* crew dispatch and final *Resolved* closure. Each event in this activity log records the new status, a precise timestamp, and optional administrative notes, giving citizens a transparent, step-by-step timeline modal that displays exactly how and when their service request is being addressed.

Third, I focused on streamlining administrative triage and workload management. To give municipal supervisors full operational oversight, I built a command dashboard equipped with real-time summary analytics cards displaying live counts for total, pending, in-progress, and resolved tickets. Additionally, I implemented dynamic, multi-parameter search logic allowing administrators to instantly filter requests by category, status, or keyword. To support offline reporting, departmental audits, and field team dispatching, I developed an in-memory CSV export utility that dynamically generates and streams downloadable report files without creating unnecessary temporary files on the server.

Fourth, I sought to establish a post-resolution quality assurance loop to bring public accountability to municipal service delivery. I designed a feedback mechanism that automatically unlocks once a service ticket is marked as *Resolved*. This feature empowers citizens to evaluate the quality of completed repairs by submitting a 1-to-5-star rating alongside qualitative feedback comments. By permanently attaching this satisfaction data to the request record, the system provides municipal leadership with measurable insights into field crew performance, community satisfaction levels, and overall service delivery standards.

2. SYSTEM ARCHITECTURE & TECH STACK

2.1 Technology Stack & Architectural Justification

In architecting the Public Service Request System, my primary design goal was to select a modern, lightweight, and highly maintainable technology stack that could support rapid development while ensuring security, responsiveness, and scalable data handling. Rather than relying on heavy monolithic frameworks, I selected a modular Python-based micro-framework paired with a flexible NoSQL document database. This combination allowed me to maintain complete control over application routing, database queries, and session security.

2.2 Backend Application Layer (Python / Flask)

I selected Python 3.13 alongside the Flask micro-framework as the core engine for the application backend. Flask's lightweight nature provided the flexibility needed to build custom middleware and decorator guards without forced boilerplate code. To enforce clean separation of concerns and maintainable code organization, I implemented a Blueprint-based architecture divided into three dedicated modules:

- auth: Handles user registration, authentication verification, and session termination.
- citizen: Manages public issue submissions, image upload handling, citizen dashboard rendering, and satisfaction feedback submissions.
- admin: Controls the administrative command center, real-time ticket triage, status log updates, analytics calculations, and CSV data exports.

2.3 Data Persistence Layer (MongoDB / PyMongo)

For database storage, I opted for MongoDB using the official PyMongo driver. A document-oriented NoSQL database was ideal for this project because public service requests require dynamic, nested data structures—specifically the embedded activity_log array for audit trails and the embedded rating object for post-resolution feedback. Storing these event arrays directly inside the primary request document eliminated the need for complex relational table joins (SQL JOINs), enabling single-document atomic updates and ensuring fast query performance when retrieving complete issue histories.

2.4 Presentation & Frontend Layer (HTML5, CSS3, Bootstrap 5)

The presentation layer was built using HTML5, CSS3, and Bootstrap 5, augmented with Bootstrap Icons. I designed a dual-theme visual hierarchy using a Deep Burgundy (#581845) and Rose Gold palette for structural elements, complemented by role-specific styling (emerald green accents for the citizen interface and slate tones for the administrator command center). Bootstrap’s responsive grid system ensured that both the citizen reporting forms and the administrative command dashboard adapt seamlessly across desktop monitors, tablets, and mobile devices.

2.4 Authentication & Security Layer (Werkzeug & Custom RBAC)

Security was integrated directly into the request pipeline. Password security is enforced using Werkzeug's cryptographic password hashing utilities (pbkdf2:sha256), ensuring that plain-text passwords are never stored in the database. State management is maintained through secure, server-side Flask Sessions. To enforce access control, I developed custom Python decorator guards (@admin_required and @citizen_required). These decorators intercept incoming requests, verify active session tokens and assigned roles, and automatically redirect unauthorized users to their correct dashboards with informative context notifications.

2.5 Reporting & In-Memory Data Streaming (csv & io.StringIO)

To give administrators offline auditing capabilities without compromising server performance, I engineered an in-memory CSV export module using Python's native csv writer and io.StringIO buffer. When an administrator requests a data export, the system queries MongoDB, processes the matching records, dynamically formats the output in memory, and streams it directly to the browser as an HTTP file attachment response. This approach avoids creating temporary CSV files on the server's local disk, reducing I/O overhead and preventing disk clutter.

2.6 Tech Stack Summary

Architecture Layer	**Component / Technology**	**Primary Purpose & Implementation Justification**
Backend Framework	Python / Flask	Micro-framework structured with Blueprints (auth, citizen, admin) for modular routing and maintainable code separation.
Data Persistence	MongoDB (PyMongo Driver)	NoSQL document database selected for schema flexibility and efficient atomic updates on nested audit logs and feedback arrays.
Frontend UI	HTML5, CSS3, Bootstrap 5	Modern, mobile-responsive layout framework providing consistent UI rendering across desktop and mobile browsers.
Iconography & Badges	Bootstrap Icons	Visual status badges, progress timeline indicators, and interactive star rating elements.
Authentication & Security	Werkzeug & Flask Sessions	Secure password hashing (pbkdf2:sha256) combined

		with custom RBAC decorator guards for role isolation.
Reporting & Export	In-Memory csv & io.StringIO	Memory-buffered streaming module generating dynamic CSV reports without generating temporary disk files.

3. CORE MODULES & IMPLEMENTATION DETAILS

3.1 Role-Based Access Control (RBAC) & Security Architecture

When architecting the security and authentication framework for the Public Service Request System, my primary objective was to ensure complete isolation between administrative capabilities and citizen workflows. In a public governance portal, preventing unauthorized privilege escalation, cross-portal session leakage, and direct object reference vulnerabilities is critical. To achieve this level of security without introducing heavy third-party dependencies, I implemented a custom, lightweight Role-Based Access Control (RBAC) system built directly into the Flask routing layer using Python function decorators.

Rather than relying solely on client-side navigation links or basic conditional checks within template views, I engineered server-side route guards that intercept every incoming HTTP request before view functions are executed. User identities and assigned system roles are securely maintained across requests using encrypted, server-side Flask sessions. Password security at registration and login is enforced using Werkzeug's cryptographic hashing utilities (pbkdf2:sha256), ensuring that user credentials are protected against database compromise.

To strictly enforce session boundaries across the application's modular blueprints, I created two primary custom decorator guards:

- @admin_required Guard: I applied this decorator to all administrative routes, including the command center dashboard, request triage update endpoints, and CSV report export functions. When a user attempts to access an administrative endpoint, the decorator first verifies whether a active user_id exists in the current session. If no session is present, it issues a protective warning flash message and redirects the user to the login screen with the administrator role tab pre-selected. If a session exists but the associated role is identified as citizen, the guard immediately blocks execution, logs an access restriction warning, and gracefully redirects the user back to their citizen dashboard rather than crashing or throwing a generic HTTP error page.
- @citizen_required Guard: I designed this decorator to protect citizen-specific views, such as new issue submission forms, media upload processing, personal dashboard history, and post-resolution satisfaction rating forms. The decorator ensures that only authenticated accounts with a citizen role can lodge requests or post feedback under their user ID. Furthermore, if a logged-in administrator intentionally or accidentally navigates to a citizen route (such as /new or /dashboard), the decorator detects the administrative session role, displays an informative notice explaining that an admin session was detected, and automatically re-routes the administrator back to the Admin Command Center.

By standardizing these decorator patterns across app/views/admin.py and app/views/citizen.py, I established an intuitive security workflow. Unauthenticated traffic is cleanly channeled to the authentication portal, while logged-in users who attempt to access cross-portal routes are automatically redirected to their designated dashboards with clear context feedback. This architecture guarantees strict role segregation, prevents URL manipulation exploits, and maintains a seamless user experience across both portals.

3.2 Administrative Triage & Analytics Command Center

To give municipal supervisors and administrative personnel total operational oversight over incoming service requests, I designed and developed a centralized Administrative Command Center. In traditional municipal workflows, administrative staff often struggle with fragmented paperwork or disorganized spreadsheets, which slows down response times and makes tracking active workloads difficult. To solve these operational challenges, I focused on building an interactive administrative dashboard (app/views/admin.py) equipped with three key capabilities: real-time metric analytics, dynamic query filtering, and on-demand report generation.

First, I implemented a Real-Time Analytics Summary Engine located at the top of the command dashboard. To provide municipal managers with immediate situational awareness, I engineered database aggregation queries that dynamically compute and render real-time request volume metrics. These summary cards display four live key performance indicators: Total Reports Submitted, Pending Review (unprocessed incoming tickets), In Progress (issues currently assigned to field repair teams), and Resolved (completed maintenance jobs). By giving administrators an instant visual snapshot of system-wide workloads, decision-makers can quickly identify operational bottlenecks and reallocate municipal repair resources to high-priority categories.

Second, I engineered a Multi-Parameter Search & Filtering Engine to help administrative personnel quickly locate specific issues out of high-volume ticket queues. To achieve this, I wrote backend query building logic using MongoDB regex pattern matching alongside PyMongo query operators. The search engine allows administrators to execute real-time text searches across multiple document fields simultaneously—including incident locations, issue categories, detailed descriptions, and citizen names. Additionally, I integrated single-click status filter controls that allow staff to isolate tickets by operational state (*Pending*, *In Progress*, or *Resolved*). Combining keyword search with categorical status filtering ensures that municipal workers can rapidly triage urgent complaints, locate specific geographic areas, and update ticket statuses without scrolling through irrelevant records.

Third, I developed a lightweight, serverless Data Export Engine to support departmental reporting, field dispatching, and administrative audits. Recognizing that municipal teams often need offline copies of service logs for field crews or weekly management reviews, I created a dedicated export endpoint (/admin/export_csv). When an administrator clicks the export trigger, the backend captures the current filtered query view, retrieves the matching MongoDB records,

and compiles the dataset using Python's native csv library and io.StringIO memory buffer. The generated CSV report includes complete ticket metadata: unique Request IDs, submission timestamps, category classification, precise incident location, issue description, current operational status, latest administrative notes, and citizen feedback ratings. By streaming the file directly to the client browser from memory, the application delivers instant, downloadable reporting capabilities without saving temporary files or creating server disk overhead.

3.3 Activity Log & Milestone Timeline Architecture

A primary complaint with traditional public service management is the lack of communication during the issue resolution lifecycle. Citizens are frequently left uninformed once an issue is submitted, unsure whether field crews have been dispatched or if the ticket has been closed without action. To solve this opacity, I engineered an automated, event-driven tracking module that generates an audit trail for every service request logged in the system.

On the backend, I updated the database update logic within the administrative blueprint (app/views/admin.py) so that every status transition automatically records a structured event log. Using MongoDB's atomic document update operators (specifically $push), whenever an administrator updates a request's state—such as moving an issue from *Pending* to *In Progress* or marking it as *Resolved*—the system appends a new event object directly into the document's embedded activity_log array. Each logged event entry captures three critical metadata attributes:

- Status Label: The newly assigned operational state (*Pending*, *In Progress*, or *Resolved*).
- Administrative Comment: An optional contextual note written by the administrator (e.g., *"Inspection team dispatched to site"* or *"Pothole sealed by public works crew #2"*).
- Formatted Timestamp: A human-readable date and time stamp (MMM DD, YYYY - HH:MM AM/PM) generated at the precise moment the status update is processed on the server.

On the frontend presentation layer, I designed an interactive, vertical Milestone Timeline Modal within the citizen dashboard (app/views/citizen.py). When a citizen views their submitted requests, they can click a "View Activity Log" button on any request card to launch a modal overlay. This modal dynamically renders the array of historical log entries in chronological order, complete with color-coded status badges (Yellow for *Pending*, Info Blue for *In Progress*, and Emerald Green for *Resolved*), clean vertical visual connectors, and corresponding administrative updates.

By linking backend atomic array updates to a clean frontend timeline modal, I created an immutable, transparent history log for every incident report. This ensures citizens can independently track the exact progression of their request at any time, eliminating the need for manual follow-up inquiries while establishing operational accountability for municipal maintenance workflows.

3.4 Citizen Satisfaction Feedback Loop Architecture

To close the operational loop between municipal service delivery and community experience, I engineered a dedicated Citizen Satisfaction Feedback Loop. In conventional civic management systems, the service workflow terminates unilaterally when an administrator or field worker marks a complaint ticket as closed in an internal database. This approach provides municipal supervisors with no objective mechanism to evaluate field performance, verify whether repairs were completed satisfactorily, or capture public sentiment regarding local infrastructure maintenance. To introduce public accountability into the system, I designed a post-resolution feedback pipeline within the citizen workflow (app/views/citizen.py).

The satisfaction feedback module operates on a strict, conditional trigger mechanism embedded in the backend logic. The system continuously monitors the operational state of each request, and only when a ticket's status transitions to Resolved does the application dynamically unlock the "Rate Service" action button on the citizen's dashboard. For active *Pending* or *In Progress* requests, the rating interface remains inaccessible, preventing premature or invalid feedback entries before maintenance work has actually been executed.

When a citizen launches the rating interface, the application opens an interactive feedback modal containing a custom-styled 1-to-5-star rating selector alongside an optional qualitative feedback text area. Citizens can evaluate the resolution quality based on criteria such as repair timeliness, work thoroughness, and administrative communication, while providing detailed narrative notes regarding their experience. Upon form submission, the backend processes the incoming input, validates that the score falls within the valid range (1–5), and executes an atomic MongoDB document update. This operation permanently attaches three dedicated fields directly to the request document:

- rating: An integer representing the assigned 1-to-5-star score.
- rating_comment: A string containing the citizen's optional feedback narrative.
- rated_at: A formatted server timestamp recording the exact time the rating was submitted.

Once saved to the database, the citizen dashboard updates dynamically. The "Rate Service" button is replaced with an immutable visual star badge (e.g., displaying yellow star icons alongside the recorded feedback note), providing the citizen with a clear record of their evaluation. On the administrative side, these satisfaction metrics are exposed alongside request records and included in dynamic CSV data exports. By capturing structured quantitative scores and qualitative commentary directly within the document schema, the feedback loop provides municipal management with data-driven insights to assess field crew efficiency, identify recurring quality issues, and ensure public service standards remain high across all municipal sectors.

4. DATABASE DESIGN & DATA MODEL

In designing the data architecture for the Public Service Request System, I selected MongoDB as the primary data persistence layer due to its flexible document-oriented schema and strong support for embedded arrays and sub-documents. Unlike rigid relational databases (RDBMS) that require complex multi-table joins (SQL JOINs) and foreign key constraints across separate tables, MongoDB allows related data such as step-by-step activity logs and post-resolution feedback to be stored atomically within a single document.

I structured the database into two primary collections: users and requests. This separation maintains clean domain isolation between user access identities and operational public service tickets, while using MongoDB's ObjectId references to maintain logical relationships between citizens and their submitted reports.

4.1 Collection: users

I designed the user's collection to manage user identity, authentication credentials, and system privilege levels. Each document in this collection represents a registered system user—either a public citizen or a municipal administrator.

To ensure security, sensitive authentication details such as plain-text passwords are never stored. Instead, I integrated Werkzeug's cryptographic hashing functions (pbkdf2:sha256) during account registration to store a non-reversible cryptographic hash. System roles (citizen or admin) are stored explicitly as string attributes, allowing backend Role-Based Access Control (RBAC) middleware to verify authorization levels during session initialization.

Document Schema (users)

JSON

```
{
  "_id": { "$oid": "671362e8a1f43c12a8b01101" },
  "full_name": "Jane Citizen",
  "email": "jane.citizen@example.org",
  "password_hash": "pbkdf2:sha256:260000$uG8xL... cryptographic hash ...",
  "role": "citizen",
  "created_at": { "$date": "2026-10-19T08:30:00.000Z" }
}
```

Schema Field Definitions

- _id: Unique BSON ObjectId auto-generated by MongoDB to identify the user record.

- full_name: String containing the full official name of the registered user.
- email: String storing the user's primary email address, serving as the unique account credential for login authentication.
- password_hash: Secure cryptographic string produced using Werkzeug's generate_password_hash utility (pbkdf2:sha256).
- role: System privilege string (citizen or admin) evaluated by custom RBAC route decorators (@admin_required, @citizen_required).
- created_at: BSON UTC timestamp tracking account registration date and time.

4.2 Collection: requests

The requests collection serves as the central operational repository for all public service tickets lodged within the portal. I architected this collection using an Embedded Data Pattern, nesting both the chronological activity history and post-resolution satisfaction feedback directly inside the parent request document.

This design choice eliminates the need to execute separate database lookups across logs or ratings tables when rendering the citizen dashboard or administrative command center. Using MongoDB's atomic update operators ($push for activity logs and $set for status updates and ratings), the backend executes real-time updates safely without risking data inconsistency.

Document Schema (requests)

JSON

```
{
  "_id": { "$oid": "67136412a1f43c12a8b01105" },
  "user_id": { "$oid": "671362e8a1f43c12a8b01101" },
  "category": "Roads & Public Works",
  "location": "Avenue 4, North Sector",
  "description": "Deep pothole blocking single-lane traffic flow near the intersection.",
  "image_filename": "6ad4ae11_potholes.png",
  "status": "Resolved",
  "admin_comment": "Resurfacing completed by public works crew #3.",
  "activity_log": [
    {
```

```
    "status": "Pending",
    "comment": "Request submitted by citizen.",
    "timestamp": "Oct 19, 2026 - 08:35 AM"
   },
   {
    "status": "In Progress",
    "comment": "Inspection team dispatched to site.",
    "timestamp": "Oct 19, 2026 - 10:15 AM"
   },
   {
    "status": "Resolved",
    "comment": "Resurfacing completed by public works crew #3.",
    "timestamp": "Oct 19, 2026 - 02:00 PM"
   }
  ],
  "rating": 5,
  "rating_comment": "Quick turnaround and thorough repair work. Thank you!",
  "rated_at": "Oct 19, 2026 - 02:45 PM",
  "created_at": { "$date": "2026-10-19T08:35:00.000Z" },
  "updated_at": { "$date": "2026-10-19T14:00:00.000Z" }
}
```


Schema Field Definitions

- _id: Unique BSON ObjectId auto-generated by MongoDB to identify the service request.
- user_id: BSON ObjectId referencing the _id of the citizen user who created the report (Foreign Key equivalent).
- category: String classification defining the infrastructure domain (e.g., *Roads & Public Works*, *Water & Sanitation*, *Street Lighting*, *Waste Management*).
- location: String providing physical location or street address details of the reported issue.

- description: Detailed text narrative describing the infrastructure defect or service disruption.
- image_filename: String holding the unique secure filename stored in the static upload directory (or null if no image was attached).
- status: Current operational state string (*Pending*, *In Progress*, or *Resolved*).
- admin_comment: String containing the most recent administrative comment provided during status triage.
- activity_log: Array of embedded sub-documents storing the chronological audit history. Each element contains:
    - status: State label at the time of the event.
    - comment: Contextual administrative note.
    - timestamp: Human-readable date and time string formatted at server execution.
- rating: Integer (1–5) representing the citizen's post-resolution evaluation score (null until ticket is resolved).
- rating_comment: Qualitative text narrative provided by the citizen during service rating (optional).
- rated_at: Formatted timestamp tracking when the satisfaction rating was submitted.
- created_at: BSON UTC timestamp recording ticket submission time.
- updated_at: BSON UTC timestamp updated automatically on every status transition.

5. USER INTERFACE & DESIGN SYSTEM

When designing the presentation layer for the Public Service Request System, my primary UI/UX objective was to create an intuitive, professional, and accessible user interface tailored to two distinct user groups: local citizens and municipal administrative staff. Civic software often suffers from cluttered layouts, confusing navigation, and poor mobile responsiveness. To avoid these pitfalls, I built the application frontend using HTML5, CSS3, and Bootstrap 5, creating a cohesive visual design system that balances aesthetic refinement with operational utility.

5.1 Palette & Global Styling

To establish a distinct brand identity, I defined a primary color palette anchored by Deep Burgundy (#581845) for top navigation bars, structural headers, and primary action buttons, complemented by soft Rose Gold accents and subtle border highlights. Deep burgundy conveys authority, stability, and professionalism appropriate for a public governance portal, while rose gold accents add visual contrast to guide user focus toward key interactive elements. Global typography relies on clean sans-serif system fonts, ensuring optimal legibility across varying display resolutions and operating systems.

5.2 Role-Specific Visual Branding

To prevent user confusion and reinforce role segregation, I established visual styling tailored specifically to each portal:

- Citizen Portal Styling: I designed the citizen interface (templates/citizen/) around accessibility, ease of use, and quick task completion. The layout utilizes spacious, elevated card containers with soft box shadows to present submitted requests cleanly. To give citizens immediate positive feedback on active tickets, I integrated emerald green highlights for completed state indicators and submission triggers. The issue reporting page features a structured multi-field form equipped with file input controls for photo attachments, ensuring citizens on smartphones can take and upload photo evidence directly from their devices.
- Admin Command Center Styling: I built the administrator interface (templates/admin/) as a high-density, professional command dashboard styled with slate gray background tones (#1e293b) and high-contrast analytical panels. The top of the page features metric summary cards providing quick quantitative counts. To enable municipal staff to scan high-volume ticket queues rapidly, I implemented a standardized status badge system:
    - Yellow Badges (bg-warning): Highlights new, unprocessed Pending requests requiring immediate administrative review.
    - Info Blue Badges (bg-info): Indicates In Progress tickets currently assigned to active field repair crews.

- Emerald Green Badges (bg-success): Marks Resolved tickets where maintenance work has been completed.

5.3 Overlay Modal Architecture

To maintain clean, uncluttered data tables on both citizen and administrative dashboards, I utilized Bootstrap 5 Modal dialogs rather than cluttering primary views with dense text rows:

- Milestone Timeline Modal: Launches an interactive overlay showing a vertical step-by-step progress trail for a specific service request. This keeps main dashboard tables focused on summary columns while giving citizens single-click access to complete activity logs.
- Service Rating Modal: Provides an isolated, pop-up dialog containing a custom-styled 1-to-5-star selector and feedback text area. This modal automatically handles form validation before dispatching the rating payload to the backend via asynchronous POST requests.

6. VERIFICATION & TESTING SUMMARY

To ensure system stability, security compliance, and data integrity across all core workflows, I conducted rigorous manual integration and functional testing throughout the development lifecycle. Given the multi-tier role structure and event-driven database updates, verifying edge cases—such as unauthorized access attempts and concurrent status modifications—was critical to preventing runtime exceptions and ensuring reliable performance.

I structured the testing process around four key verification scenarios:

1. RBAC Security & Privilege Isolation	Authenticated as a standard citizen user, I manually manipulated the browser URL to access restricted administrative endpoints (/admin/dashboard).	The custom @admin_required decorator intercepted the request, blocked execution, and cleanly redirected the user back to /dashboard with an informative warning notification.	**PASSED**
2. Activity Log Event-Driven Tracking	Logged into the administrative command center and transitioned an active service ticket status from *Pending* to *In Progress* with an optional admin note.	The backend executed an atomic MongoDB $push operation, instantly appending a structured audit event containing the new status, comment, and timestamp to the document's activity_log array.	**PASSED**
3. Filtered CSV Data Export	Applied keyword search and status filters on the admin dashboard, then triggered the "Export CSV" function.	The server compiled the matching MongoDB query results into an in-memory buffer, successfully streaming a formatted .csv file download containing only the filtered dataset without disk clutter.	**PASSED**
4. Citizen Satisfaction Feedback Loop	Logged into a citizen account, opened a resolved request record, and submitted	The backend verified the request status was *Resolved*, saved the rating and feedback	**PASSED**

	a 5-star rating with qualitative feedback.	comment to MongoDB, closed the modal overlay, and instantly replaced the action button with a visual star badge.	

7. FUTURE WORK & SCALABILITY ROADMAP

While the Public Service Request System successfully fulfills its core objective of bridging the gap between citizens and municipal authorities through transparent tracking and administrative triage, there are several strategic technical enhancements I plan to implement in future iterations to scale the application for enterprise municipal deployment:

1. Automated Notification Engine (SMTP & SMS Integration): Currently, citizens must log into the portal to check the status of their requests via the milestone timeline modal. In a future update, I plan to integrate an asynchronous notification service (such as Flask-Mail for SMTP email alerts or the Twilio API for SMS notifications) that automatically dispatches real-time alerts to citizens whenever an administrator updates their ticket status or posts a repair note.
2. Geospatial Mapping & Pin-Drop Location Selection: Presently, the system captures location data via standard text input fields (e.g., street names or sector descriptions). To improve field crew dispatch accuracy, I intend to integrate an interactive mapping library—such as Leaflet.js paired with OpenStreetMap or Mapbox APIs. This will allow citizens to drop an interactive pin on a map to log precise GPS coordinates for infrastructure defects.
3. Multi-Departmental Routing & Sub-Admin Workloads: The current architecture utilizes a centralized administrative queue. As municipal operations scale, managing all infrastructure issues under a single admin panel can become a bottleneck. Future enhancements will introduce granular departmental roles (e.g., *Water & Sanitation*, *Roads & Public Works*, *Electrical & Street Lighting*, and *Waste Management*), allowing super-admins to route incoming issues directly to specialized maintenance departments and sub-administrators.

8. CONCLUSION

Through the design, architecture, and implementation of the Public Service Request System, I successfully demonstrated how a lightweight, modern technology stack can effectively solve real-world challenges in municipal governance and public service delivery. By pairing a modular Python and Flask backend with a flexible MongoDB document database, I built a responsive web platform that eliminates the operational opacity of traditional civic complaint workflows.

The integration of robust Role-Based Access Control (RBAC) guaranteed strict security isolation between citizens and administrative personnel, while the automated activity logging audit trail and milestone timeline modals transformed issue tracking into a transparent, event-driven experience. Furthermore, the implementation of an administrative command center with in-memory CSV reporting and a post-resolution citizen satisfaction feedback loop established a complete accountability lifecycle. Ultimately, this project serves as a functional, scalable software foundation capable of streamlining municipal operations, improving response times, and restoring public trust through digital transparency.